МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних технологій

ЛАБОРАТОРНА РОБОТА №5

З дисципліни: «Об’єктно-орієнтоване та подійно-орієнтоване програмування»
На тему: «Особливості та призначення інтерфейсів та принципу успадкування»

Роботу виконав:
студент II курсу групи АД-251
спеціальності F6
Горбунова А.А.

Роботу перевірив:
к т.н., доцент Рудніченко М.Д.

Оцінка: _____

ДУОП, 2026

1 МЕТА РОБОТИ ТА ЗАВДАННЯ

1.1 Мета лабораторної роботи

Метою лабораторної роботи є отримати практичні навички зі створення і використання абстрактних класів та інтерфейсів, засвоїти принципи успадкування класів у мові програмування Java, навчитися створювати ієрархії класів, перевизначати методи базових класів, реалізовувати методи інтерфейсів та використовувати поліморфізм під час роботи з об’єктами. У процесі виконання роботи необхідно:

1.2 Загальне завдання (Практична частина)

Під час виконання лабораторної роботи необхідно створити Java-програму, у якій буде продемонстровано використання успадкування, абстрактного класу, інтерфейсу та поліморфізму.:

1) Створення базового абстрактного класу;
2) Створення дочірніх класів;
3) Створення інтерфейсу;
4) Створення об’єктів;
5) Демонстрація поліморфізму;
6) Робота з інтерфейсом;
7) Розширення програми: Після виконання базової частини необхідно адаптувати приклад до індивідуального завдання. Програма повинна містити:

а) щонайменше один абстрактний клас;
б) щонайменше два дочірні класи;
в) щонайменше один інтерфейс;
г) реалізацію інтерфейсу конкретним класом;
г) перевизначення методів;
д) конструктори базового та дочірніх класів;
е) використання `super`;

є) демонстрацію поліморфізму;

ж) створення та використання об’єктів у методі `main()`.

1.3 Індивідуальне завдання. Варіант №7. Комп’ютерна техніка.

1) Створити абстрактний клас `ComputerDevice` з полями `brand` та `model`;
2) Створити класи `Laptop` та `Desktop`;
3) Додати інтерфейс `Connectable` з методом `connect()`.

1.4 Додаткові вимоги до виконання:

а) Індивідуальне завдання необхідно реалізувати мовою Java.

б) Основний об’єкт предметної області повинен бути представлений окремим класом. Для побудови ієрархії класів необхідно використовувати успадкування за допомогою extends.

в) Необхідно продемонструвати використання абстрактного класу та щонайменше одного абстрактного методу.

г) Дочірні класи повинні перевизначати абстрактні або звичайні методи базового класу з використанням анотації `@Override`.

ґ) Для реалізації додаткової поведінки необхідно створити інтерфейс та реалізувати його в одному або декількох класах за допомогою `implements`.

д) Якщо це передбачено індивідуальним завданням, необхідно продемонструвати реалізацію декількох інтерфейсів одним класом.

е) У конструкторах дочірніх класів необхідно продемонструвати використання `super()` для передавання даних до конструктора базового класу.

є) Для демонстрації поліморфізму необхідно створити змінну або масив базового типу, який містить об’єкти різних дочірніх класів.

ж) Програмний код повинен бути структурованим та зрозумілим. Назви класів, методів і змінних повинні відповідати їхньому призначенню.

з) Необхідно забезпечити працездатність програми та відсутність помилок компіляції.

и) У звіті необхідно навести програмний код основних класів, результати виконання програми та коротко пояснити, де саме реалізовано успадкування, абстракцію, інтерфейс і поліморфізм.

1.5 Оформити звіт і відповісти на контрольні питання.

2 СТИСЛІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Під час виконання лабораторної роботи, а саме проєктування системи було використано такі фундаментальні концепції об’єктно-орієнтованого програмування:

– **Успадкування (`extends`):** Механізм, що дозволяє створювати нові класи на основі вже існуючих, успадковуючи їхні поля та методи, забезпечуючи повторне використання коду. В Java підтримується лише поодиноке успадкування для класів.

– **Абстрактні класи (`abstract`):** Класи, призначені для створення спільної основи. Вони можуть містити поля, конструктори, звичайні методи та абстрактні методи без реалізації, які обов’язково мають бути перевизначені в дочірніх класах.

– **Інтерфейси (`interface` / `implements`):** Визначають певний набір методів (контракт поведінки) без реалізації. Клас може реалізовувати декілька інтерфейсів одночасно, що компенсує відсутність множинного успадкування класів.

– **Поліморфізм:** Здатність програми працювати з об’єктами різних дочірніх класів через спільний базовий тип (суперклас або інтерфейс), забезпечуючи динамічне зв’язування методів на етапі виконання.

3 ОПИС ПРОЦЕСУ СТВОРЕННЯ АРХІТЕКТУРИ ПРОГРАМНОГО КОМПЛЕКСУ

Головною інженерною особливістю розробленого додатка є об'єднання двох різних за семантикою практичних задач в одному виконуваному файлі `Main.java` під керуванням віртуальної машини Java (JVM). Процес декомпозиції та побудови коду виконувався у кілька послідовних етапів з дотриманням принципу розділення відповідальності.

Для розуміння того, як буде виглядати взаємодія об'єктів у додатку, та облегшення процесу створення (написання) коду, спочатку, за допомогою застосунку `Drawio`, побудуэмо UML-дыаграму.

Нижче на малюнку 3.1 наведено скріншот UML-діаграми для лабораторної роботи яка складається з двух окремих завдань, які об'єднані головним файлом `Main.java`

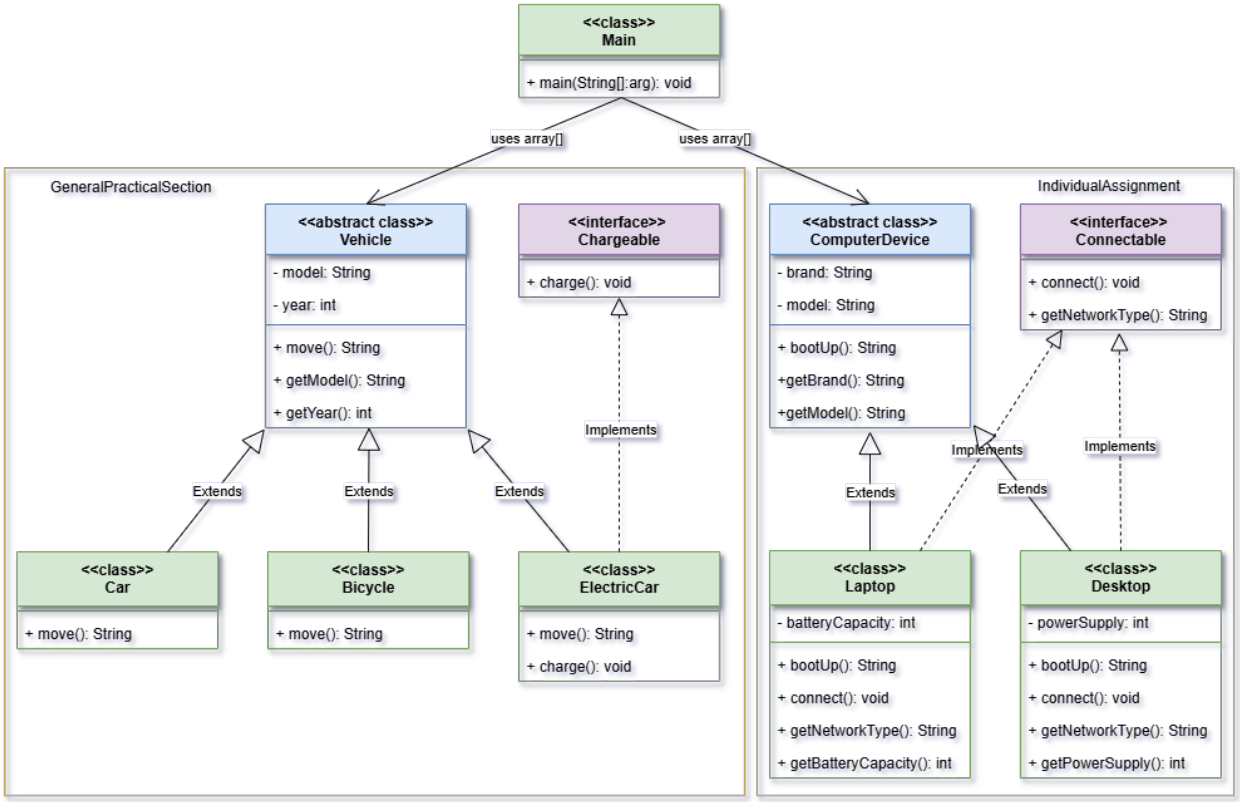


Рисунок 3.1 – UML-діаграма класів лабораторної роботи №5

3.1 Компонентний розбір архітектури системи

Структура додатка чітко розмежована на два логічні пакети (packages), що запобігає змішуванню бізнес-контекстів і забезпечує чистоту проєктування:

1) **Пакет `GeneralPracticalSection` (Загальний приклад):**

– `Vehicle` — базовий абстрактний клас, що задає інкапсульований стан (`model`, `year`), відкриті геттери для доступу до них та абстрактний функціональний контракт у вигляді методу `move(): String`.

– `Car`, `Bicycle` — конкретні класи-спадкоємці, які перевизначають метод `move()` для повернення специфічних рядків стану.

– `Chargeable` — функціональний інтерфейс-маркер поведінки із методом `charge(): void`.

– `ElectricCar` — дочірній клас, який демонструє поєднання вертикального успадкування (`extends Vehicle`) та горизонтального розширення можливостей через імплементацію інтерфейсу (`implements Chargeable`).

2) **Пакет `IndividualAssignment` (Індивідуальний Варіант №7):**

– `ComputerDevice` — кореневий абстрактний клас ієрархії обчислювальної техніки з полями `brand`, `model` та абстрактним методом ініціалізації операційної системи `bootUp(): String`.

– `Connectable` — мережевий функціональний інтерфейс, який вводить контракт підключення `connect(): void` та поліморфний метод опису фізичного каналу зв'язку `getNetworkType(): String`.

– `Laptop`, `Desktop` — кінцеві конкретні класи сутностей. Обидва класи одночасно розширюють базовий пристрій та імплементують мережевий інтерфейс, надаючи унікальні геттери для власних прихованих полів (`batteryCapacity` та `powerSupply` відповідно).

3.2 Аналіз типів зв'язків між елементами діаграми

Для побудови коректної об'єктної моделі на схемі було використано три фундаментальні типи зв'язків стандарту UML:

– **Успадкування / Генералізація (Generalization):** Позначено суцільною лінією з незафарбованим трикутним покажчиком на кінці, спрямованим від дочірніх класів до суперкласів (`Car` → `Vehicle`, `Laptop` → `ComputerDevice`). Символізує відношення *«is-a»* (є цим об'єктом).

– **Реалізація / Імплементація (Realization):** Позначено пунктирною лінією з незафарбованим трикутним покажчиком, спрямованим від конкретного класу до інтерфейсу (`ElectricCar Chargeable`, `Desktop Connectable`). Визначає готовність класу виконувати контракт поведінки інтерфейсу.

– **Залежність / Використання (Dependency):** Позначено простою стрілкою від керуючого класу `Main` до масивів базових абстрактних типів. Вказує на те, що зміни в сигнатурах методів `Vehicle` або `ComputerDevice` можуть вплинути на логіку роботи точки входу, проте самі класи сутностей нічого не знають про існування класу `Main` (досягнуто низьку зв'язаність системи).

3.3 Ізоляція бізнес-логіки сутностей від CLI-середовища

Для забезпечення гнучкості архітектури було вирішено повністю відмовитися від використання прямих викликів `System.out.println()` всередині методів класів, що описують сутності.

– В ієрархії **Vehicle** абстрактний метод `move()` було спроєктовано з сигнатурою, що повертає тип `String`. Таким чином, класи `Car` та `Bicycle` лише формують та віддають текстовий опис своєї дії, не зачіпаючи потоки виведення.

– В ієрархії **ComputerDevice** метод ініціалізації ОС `bootUp()` аналогічно повертає `String`. Для доступу до внутрішнього стану прихованих полів (`model`,

brand, batteryCapacity, powerSupply) було розроблено відкриті геттери, що дозволило зберегти сувору інкапсуляцію даних.

3.4 Проєктування та розширення інтерфейсних контрактів

Інтерфейси в системі виступають як маркери додаткової поведінки, що застосовуються лише до певних вузлів ієрархії:

– Інтерфейс **Chargeable** інтегрується виключно в клас ElectricCar, вказуючи, що цей автомобіль потребує зарядного пристрою.

– Мережевий інтерфейс **Connectable** за умовою варіанта реалізують обидва класи комп'ютерної техніки (Laptop та Desktop). Однак фізична природа підключення у них різна. Щоб точка входу могла поліморфно дізнатися тип мережі, в контракт Connectable було введено метод getNetworkType(): String, який повертає "Wi-Fi" для портативних пристроїв та "Ethernet" для стаціонарних ПК.

3.5 Синхронізація обробки даних у Main.java

Клас Main.java виступає в ролі центрального контролера та єдиного середовища представлення даних. Виконання двох завдань реалізовано через послідовні незалежні цикли обходу поліморфних масивів:

1) **Блок загального завдання:** Створюється масив посилань Vehicle[], куди поміщаються об'єкти автомобіля, велосипеда та електрокара. Під час ітерації додаток динамічно перевіряє підтримку інтерфейсу Chargeable за допомогою оператора instanceof і виводить статус зарядки.

2) **Блок індивідуального варіанта:** Створюється масив посилань ComputerDev який містить реальні екземпляри ноутбуків та десктопів. Головний клас зчитує геттери бренду та моделі, динамічно перевіряє наявність Connectable для отрима-

ння типу мережі, а також застосовує безпечне низхідне приведення типів для отримання ємності батареї або потужності БЖ.

Окрім циклів, у `Main.java` продемонстровано третій важливий аспект ООП: створення об’єкта `HP EliteBook` через інтерфейсну змінну типу `Connectable officeLaptop = new Laptop(...)`. Це наочно ілюструє приховування деталей реалізації, коли програміст бачить лише методи підключення до мережі, але об’єкт у кучі залишається повноцінним ноутбуком, який згодом безпечно приводиться до базового класу для відображення в загальній структурі.

4 ЛІСТИНГ ПРОГРАМНОГО КОДУ

У цьому розділі наведено повний вихідний код розробленого програмного комплексу. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

4.1 Клас Main.java.

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Головний керуючий клас програми (Точка входу).
 * Виконує координацію роботи обох секцій лабораторної роботи та демонструє
 * побудову динамічних CLI-таблиць на основі об'єктних моделей.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Main {

    /**
     * Точка входу в систему. Формує структурований псевдотабличний лог.
     *
     * @param args аргументи командного рядка CLI
     */
    public static void main(String[] args) {
        System.out.print("=== Лабораторна робота №5 ===\n");
        System.out.print("Тема: Особливості призначення інтерфейсів "
                         +"та принципу успадкування.\n\n");

        // ======================================================================
        // 1. ЗАГАЛЬНА ПРАКТИЧНА ЧАСТИНА (Транспортні засоби)
        // ======================================================================
        System.out.print("--- 1. Демонстрація загального прикладу (Транспортні засоби). ---\n");
        // Генерація рядка-роздільника фіксованої довжини
        String vSeparator = "+------------"
                            +"+--------------"
                            +"+------"
                            +"+----------"
                    +"+-----------------------------------------------------------------------+";

        // Поліморфний масив: зберігає посилання на базовий абстрактний клас Vehicle,
        // але ініціалізований конкретними об'єктами дочірніх класів
        Vehicle[] vehicles = {
            new Car("Toyota", 2022),
            new Bicycle("Trek", 2023),
            new ElectricCar("Tesla", 2024)
        };

        // Виведення шапки першої таблиці
        System.out.println(vSeparator);
        // Специфікатори типу %-10s задають ширину колонки та вирівнювання по лівому краю
        System.out.printf("| %-10s | %-12s | %-4s | %-8s | %-67s |%n",
                          "Тип", "Марка/Модель", "Рік", "Зарядка", "Поточний стан / Дія");
        System.out.println(vSeparator);

        // Ітерація по масиву транспортних засобів
        for (Vehicle v : vehicles) {
```

```
61              // Визначення текстового маркера типу на основі RTTI (Run-Time Type Information)
62              String type = "Автомобіль";
63              if (v instanceof Bicycle) type = "Велосипед";
64              if (v instanceof ElectricCar) type = "Електрокар";
65
66              // Безпечна перевірка реалізації інтерфейсу Chargeable без виклику void-методу.
67              // Якщо об'єкт підтримує інтерфейс — виводимо "+", інакше "-"
68              String chargeStatus = (v instanceof Chargeable) ? "+" : "-";
69
70              // Поліморфний виклик методу v.move(). Точна реалізація визначається JVM
71              // динамічно під час виконання програми
72              System.out.printf("| %-10s | %-12s | %-4d | %-8s | %-67s |%n",
73                      type, v.getModel(), v.getYear(), "   " + chargeStatus, v.move());
74          }
75          // Закриття таблиці
76          System.out.println(vSeparator);
77          System.out.print("\n");
78
79          // ========================================================================
80          // 2. ІНДИВІДУАЛЬНЕ ЗАВДАННЯ: ВАРІАНТ №7 (Комп'ютерна техніка)
81          // ========================================================================
82          System.out.print("--- 2. Демонстрація індивідуального завдання. Варіант №7"
83                          +" (Комп'ютерна техніка). ---\n");
84          //Генерація лінійного роздільника для другої таблиці
85          String dSeparator = "+------------"
86                              +"+-------------"
87                              +"+-----------------"
88                              +"+------------"
89                              +"+-----------------------------------------------------------+";
90
91          // Створення масиву обчислювальної техніки. Демонстрація абстракції:
92          // ми працюємо з абстрактним типом ComputerDevice, приховуючи деталі реалізації
93          ComputerDevice[] devices = {
94              new Laptop("Lenovo", "ThinkPad T470s", 4600),
95              new Desktop("ASUS", "ROG Strix", 750),
96              new Laptop("Apple", "MacBook Pro", 5800)
97          };
98
99          // Виведення шапки другої таблиці
100         System.out.println(dSeparator);
101         System.out.printf("| %-10s | %-11s | %-15s | %-10s | %-55s |%n",
102                         "Тип", "Виробник", "Модель", "Мережа", "Статус ОС (bootUp)");
103         System.out.println(dSeparator);
104
105         // Поліморфний обхід масиву комп'ютерних пристроїв
106         for (ComputerDevice cd : devices) {
107             // Тернарний оператор для визначення базового типу пристрою
108             String typeLabel = (cd instanceof Laptop) ? "Ноутбук" : "Десктоп";
109             String netLabel = " ";
110
111             // Низхідне приведення після обов'язкової безпечної перевірки instanceof.
112             // Потрібно для доступу до методу getNetworkType(), якого немає в
113             // абстрактному ComputerDevice
114             if (cd instanceof Connectable) {
115                 netLabel = ((Connectable) cd).getNetworkType();
116             }
117
118             // Формуємо розширений статус з полями [АКБ], [БЖ]
119             String statusWithDetails = cd.bootUp();
120             if (cd instanceof Laptop) {
121                 statusWithDetails += " [АКБ: " + ((Laptop) cd).getBatteryCapacity() + " мАг]";
122             } else if (cd instanceof Desktop) {
123                 statusWithDetails += " [БЖ: " + ((Desktop) cd).getPowerSupply() + " Вт]";
124     }
125
126             // Динамічне зв'язування: виклик cd.bootUp() виконає метод конкретного
127             // Laptop або Desktop
128             System.out.printf("| %-10s | %-11s | %-15s | %-10s | %-55s |%n",
129                     typeLabel, cd.getBrand(), cd.getModel(), netLabel, statusWithDetails);
130         }
131
132         // Демонстрація поліморфізму через інтерфейсне посилання.
133         //Створення об'єкта, де типом посилання є інтерфейс Connectable,
134         // а типом об'єкта в кучі (Heap) — конкретний Laptop.
135         // Це обмежує видимість об'єкта виключно методами самого інтерфейсу.
136         Connectable officeLaptop = new Laptop("HP", "EliteBook", 5000);
137         // Для включення цього об'єкта в загальну таблицю комп'ютерної техніки,
138         // ми перевіряємо, чи є він спадкоємцем базового класу ComputerDevice
139         if (officeLaptop instanceof ComputerDevice) {
140             ComputerDevice polyDevice = (ComputerDevice) officeLaptop;
141             Laptop polyLaptop = (Laptop) officeLaptop; // для доступа к батарее
142
```

```
143             String statusWithDetails = polyDevice.bootUp() + " [АКБ: "
144                                                            + polyLaptop.getBatteryCapacity()
145                                                            + " мАг]";
146
147             System.out.printf("| %-10s | %-11s | %-15s | %-10s | %-55s |%n",
148                     "* Ноутбук", polyDevice.getBrand(), polyDevice.getModel(),
149                     officeLaptop.getNetworkType(), statusWithDetails);
150         }
151
152         // Закриття таблиці та виведення академічної сноски-аннотації
153         System.out.println(dSeparator);
154         System.out.println("[АННОТАЦІЯ] * — Об'єкт створено через інтерфейсний тип посилання "
155                                           +"(Connectable dev = new Laptop(...)).");
156         System.out.println("               Демонстрація динамічного поліморфізму та доступу "
157                                           +"виключно до методів інтерфейсу.");
158     }
159 }
```

4.2 Клас Vehicle.java (Загальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Базовий абстрактний клас для секції загальної практики (GeneralPracticalSection).
 * Надає базову структуру для всіх транспортних засобів.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public abstract class Vehicle {
    /** Модель транспортного засобу */
    protected String model;
    /** Рік випуску */
    protected int year;

    /**
     * Конструктор для ініціалізації загальних характеристик транспортного засобу.
     *
     * @param model марка або модель
     * @param year рік виробництва
     */
    public Vehicle(String model, int year) {
        this.model = model;
        this.year = year;
    }

    /**
     * Абстрактний метод руху, що повертає текстовий статус поточного стану.
     *
     * @return лаконічний опис дії транспортного засобу
     */
    public abstract String move();

    /**
     * Повертає назву моделі або марки транспортного засобу.
     *
     * @return рядок із назвою моделі
     */
    public String getModel() {
        return model;
    }

    /**
     * Повертає рік випуску транспортного засобу.
     *
     * @return чотиризначний рік випуску
     */
    public int getYear() {
        return year;
    }
}
```

4.3 Інтерфейс Chargeable.java (Загальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Інтерфейс для пристроїв, що підтримують заряджання.
 *
 * @author Анастасія Горбунова
 * @version 5.0 15 Oct 2026
 */
public interface Chargeable {
    void charge();
}
```

4.4 Клас Car.java (Загальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Car, що розширює Vehicle (Extends).
 * Представляє класичний автомобіль.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Car extends Vehicle {
    /**
     * Конструктор класу Car. Передає параметри базовому класу.
     *
     * @param model марка авто
     * @param year рік випуску
     */
    public Car(String model, int year) {
        super(model, year);
    }

    /**
     * Перевизначений метод руху автомобіля.
     *
     * @return рядок із описом руху автомобіля
     */
    @Override
    public String move() {
        return "Рухається загальною дорогою.";
    }
}

```

4.5 Клас Bicycle.java (Загальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Bicycle, що розширює Vehicle (Extends).
 * Представляє велосипед.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Bicycle extends Vehicle {
    /**
     * Конструктор класу Bicycle. Передає параметри базовому класу.
     *
     * @param model марка велосипеда
     * @param year рік випуску
     */
    public Bicycle(String model, int year) {
        super(model, year);
    }

    /**
     * Перевизначений метод руху велосипеда.
     *
     * @return рядок із описом руху велосипедною доріжкою
     */
    @Override
    public String move() {
        return "Рухається велосипедною доріжкою.";
    }
}

```

4.6 Клас ElectricCar.java (Загальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Клас Електромобіль. Поєднує успадкування Vehicle (extends)
 * та реалізацію Chargeable (implements).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class ElectricCar extends Vehicle implements Chargeable {
    /**
     * Конструктор класу ElectricCar. Передає параметри базовому класу.
     *
     * @param model модель електромобіля
     * @param year рік випуску
     */
    public ElectricCar(String model, int year) {
        super(model, year);
    }

    /**
     * Перевизначений метод руху електромобіля.
     *
     * @return рядок із описом руху через електродвигун
     */
    @Override
    public String move() {
        return "Рухається за допомогою електродвигуна.";
    }

    /**
     * Реалізація методу інтерфейсу Chargeable.
     * Виконує контракт інтерфейсу без зміни повертаємого типу.
     */
    @Override
    public void charge() {
        // Контрактний метод інтерфейсу
    }
}
```

4.7 Клас ComputerDevice.java (Індивідуальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Базовий абстрактний клас, що описує обчислювальну комп'ютерну техніку
 * для секції індивідуального завдання. Варіант 7. (IndividualAssignment).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public abstract class ComputerDevice {
    /** Торгова марка (бренд) пристрою */
    protected String brand;
    /** Конкретна модель пристрою */
    protected String model;

    /**
     * Конструктор абстрактного класу для ініціалізації базових полів.
     *
     * @param brand бренд виробника
     * @param model назва моделі
     */
    public ComputerDevice(String brand, String model) {
        this.brand = brand;
        this.model = model;
    }

    /**
     * Абстрактний метод для ініціалізації завантаження операційної системи.
     *
     * @return лаконічний статус завантаження ОС у вигляді рядка
     */
    public abstract String bootUp();

        /**
     * Повертає торгову марку (бренд) обчислювального пристрою.
     *
     * @return рядок із назвою бренду виробника
     */
    public String getBrand() {
        return brand;
    }

    /**
     * Повертає конкретну модель обчислювального пристрою.
     *
     * @return рядок із назвою моделі техніки
     */
    public String getModel() {
        return model;
    }
}
```

4.8 Інтерфейс Connectable.java (Індивідуальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Інтерфейс, що визначає поведінку пристроїв, здатних підключатися до мережі.
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public interface Connectable {
    /**
     * Абстрактний метод для встановлення мережевого з'єднання.
     */
    void connect();

    /**
     * Повертає тип мережевого інтерфейсу пристрою.
     *
     * @return назва типу підключення (Wi-Fi, Ethernet тощо)
     */
    String getNetworkType();
}
```

4.9 Клас Laptop.java (Індивідуальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Laptop. Поєднує успадкування ComputerDevice (extends)
 * та реалізує інтерфейс Connectable (implements).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Laptop extends ComputerDevice implements Connectable {
    /** Ємність вбудованої акумуляторної батареї у мАг */
    private final int batteryCapacity;

    /**
     * Конструктор класу Laptop. Викликає конструктор суперкласу через super().
     *
     * @param brand бренд пристрою
     * @param model модель ноутбука
     * @param batteryCapacity ємність батареї
     */
    public Laptop(String brand, String model, int batteryCapacity) {
        super(brand, model);
        this.batteryCapacity = batteryCapacity;
    }

    /**
     * Переозначений метод завантаження ноутбука.
     *
     * @return скорочений статус швидкого старту
     */
    @Override
    public String bootUp() {
        return "Швидкий запуск (Режим сну)";
    }

    /**
     * Реалізація методу інтерфейсу Connectable для підключення.
     */
    @Override
    public void connect() {
        // Логіка підключення об'єкта (внутрішній стан)
    }

    /**
     * Повертає бездротовий тип мережі ноутбука.
     *
     * @return рядок "Wi-Fi"
     */
    @Override
    public String getNetworkType() {
        return "Wi-Fi";
    }

    public int getBatteryCapacity() {
        return batteryCapacity;
    }
}
```

4.10 Клас Desktop.java (Індивідуальне завдання).

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Дочірній клас Desktop. Поєднує успадкування ComputerDevice (extends)
 * та реалізує інтерфейс Connectable (implements).
 *
 * @author Анастасія Горбунова
 * @version 5.0 23 Oct 2026
 */
public class Desktop extends ComputerDevice implements Connectable {
    /** Потужність блоку живлення в Ватах */
    private final int powerSupply;

    /**
     * Конструктор класу Desktop. Передає параметри базовому класу.
     *
     * @param brand бренд пристрою
     * @param model модель системного блоку
     * @param powerSupply потужність БП
     */
    public Desktop(String brand, String model, int powerSupply) {
        super(brand, model);
        this.powerSupply = powerSupply;
    }

    /**
     * Перевизначений метод завантаження стаціонарного ПК.
     *
     * @return скорочений статус повного старту
     */
    @Override
    public String bootUp() {
        return "Повне завантаження (Перевірка живлення)";
    }

    /**
     * Реалізація методу інтерфейсу Connectable для кабельного підключення.
     */
    @Override
    public void connect() {
        // Логіка дротового підключення об'єкта
    }

    /**
     * Повертає дротовий тип мережі десктопа.
     *
     * @return рядок "Ethernet"
     */
    @Override
    public String getNetworkType() {
        return "Ethernet";
    }

    public int getPowerSupply() {
        return powerSupply;
    }
}
```

5 РЕЗУЛЬТАТИ ВИКОНАННЯ РОБОТИ

Для підтвердження коректності роботи розробленої ієрархії класів, перевірки працездатності абстрактних методів та демонстрації динамічного поліморфізму через інтерфейсні типи посилань було виконано компіляцію та запуск коду. Компіляція та виконання програми здійснювалися в терміналі PowerShell середовища розробки VS Code з примусовим встановленням кодування UTF-8.

На рисунку 5.1 представлено скріншот вікна терміналу з результатами виконання програми.

```
--- РЕЗУЛЬТАТ ВВИКОНАННЯ ---
=== Лабораторна робота №5 ===
Тема: Особливості призначення інтерфейсів та принципу успадкування.

--- 1. Демонстрація загального прикладу (Транспортні засоби). ---
+------------+--------------+------+----------+-----------------------------------------------------------------------+
| Тип        | Марка/Модель | Рік  | Зарядка  | Поточний стан / Дія                                                   |
+------------+--------------+------+----------+-----------------------------------------------------------------------+
| Автомобіль | Toyota       | 2022 |    -     | Рухається загальною дорогою.                                          |
| Велосипед  | Trek         | 2023 |    -     | Рухається велосипедною доріжкою.                                      |
| Електрокар | Tesla        | 2024 |    +     | Рухається за допомогою електродвигуна.                                |
+------------+--------------+------+----------+-----------------------------------------------------------------------+

--- 2. Демонстрація індивідуального завддання. Варіант №7 (Комп'ютерна техніка). ---
+------------+-------------+-----------------+------------+-----------------------------------------------------------+
| Тип        | Виробник    | Модель          | Мережа     | Статус ОС (bootUp)                                        |
+------------+-------------+-----------------+------------+-----------------------------------------------------------+
| Ноутбук    | Lenovo      | ThinkPad T470s  | Wi-Fi      | Швидкий запуск (Режим сну) [АКБ: 4600 мАг]                |
| Десктоп    | ASUS        | ROG Strix       | Ethernet   | Повне завантаження (Перевірка живлення) [БЖ: 750 Вт]      |
| Ноутбук    | Apple       | MacBook Pro     | Wi-Fi      | Швидкий запуск (Режим сну) [АКБ: 5800 мАг]                |
| * Ноутбук  | HP          | EliteBook       | Wi-Fi      | Швидкий запуск (Режим сну) [АКБ: 5000 мАг]                |
+------------+-------------+-----------------+------------+-----------------------------------------------------------+
[АННОТАЦІЯ] * – Об'єкт створено через інтерфейсний тип посилання (Connectable dev = new Laptop(...)).
              Демонстрація динамічного поліморфізму та доступу виключно до методів інтерфейсу.
```

Рисунок 5.1 – Скріншот результатів виконання програми в терміналі VS Code завдання лабораторної роботи №5

Завдяки винесенню логіки форматування в головний клас `Main.java` було реалізовано принцип розділення відповідальності. Дочірні класи не містять елементів CLI-розмітки, повертаючи виключно інформаційні рядки стану, що дозволяє гнучко масштабувати систему та змінювати формат представлення даних без рефакторингу бізнес-логіки ієрархії.

6 ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1) **Що таке успадкування в об'єктно-орієнтованому програмуванні?** Це фундаментальний принцип ООП, який дозволяє створювати новий (дочірній) клас на основі вже існуючого (батьківського), автоматично переймаючи його захищені та публічні поля і методи для повторного використання коду та розширення функціональності.

2) **Яке ключове слово використовується для успадкування класів у Java?** Для реалізації успадкування класів у мові програмування Java використовується ключове слово `extends`.

3) **Чим відрізняється базовий клас від дочірнього?** Базовий (батьківський) клас описує загальні універсальні характеристики сутностей, тоді як дочірній (похідний) клас конкретизує поведінку, додає власні унікальні поля/методи та перевизначає логіку суперкласу.

4) **Чи може клас у Java успадковувати декілька класів одночасно?** Ні, в Java реалізовано лише поодиноке успадкування для класів (конструкція з кількома батьківськими класами не підтримується), проте один клас може реалізовувати декілька інтерфейсів одночасно.

5) **Для чого використовується ключове слово super?** Ключове слово `super` використовується для звернення до елементів батьківського класу: для виклику його конструктора на початку конструктора дочірнього класу (наприклад, `super(brand, model)`) або для виклику оригінальних методів суперкласу.

6) **Що таке перевизначення методу?** Це операція, за якої дочірній клас змінює внутрішню реалізацію (тіло методу) успадкованого від батьківського класу методу при повному збереженні його імені, аргументів та типу повертаємого значення.

7) **Для чого використовується анотація @Override?** Вона рекомендується для позначення перевизначеного методу та вказує компілятору Java на перевірку відповідності сигнатур. Це захищає від помилок друку — якщо сигнатури методів базового та дочірнього класів не збігаються, компілятор видасть помилку.

8) **Що таке абстрактний клас?** Це клас, оголошений із ключовим словом `abstract`, який призначений для створення спільної основи для інших класів. Він може містити як звичайні методи з реалізацією, так і абстрактні методи без тіла.

9) **Чи можна створити безпосередньо об'єкт абстрактного класу?** Ні, створити безпосередньо об'єкт абстрактного класу неможливо (наприклад, виклик `new ComputerDevice()` призведе до помилки компіляції).

10) **Що таке абстрактний метод?** Це метод, який оголошується без тіла (без конкретної реалізації, завершується крапкою з комою), задаючи лише сигнатуру, яку всі конкретні дочірні класи обов'язково повинні реалізувати.

11) **Які елементи може містити абстрактний клас?** Абстрактний клас може містити поля (змінні стану), конструктори, звичайні методи з реалізацією, а також абстрактні методи без реалізації.

12) **Що таке інтерфейс у Java?** Це повністю абстрактна конструкція, створена за допомогою ключового слова `interface`, яка визначає певний набір методів (контракт поведінки), що повинні бути реалізовані класом.

13) **Яке ключове слово використовується для реалізації інтерфейсу?** Клас реалізує інтерфейс за допомогою ключового слова `implements`.

14) **Чи може один клас реалізовувати декілька інтерфейсів?** Так, однією з важливих переваг інтерфейсів є можливість реалізувати декілька інтерфейсів одним конкретним класом через кому.

15) **У чому полягає основна відмінність між абстрактним класом та інтерфейсом?** Абстрактний клас доцільно використовувати тоді, коли класи мають спільний стан, поля та частково спільну реалізацію (зв'язок сутностей). Інтерфейс доцільно використовувати для визначення певної можливості або набору операцій, які можуть бути реалізовані різними, навіть не пов'язаними між собою класами.

16) **Що таке поліморфізм?** Це принцип ООП, який дає змогу працювати з об'єктами різних дочірніх класів через спільний базовий тип (суперклас або інтерфейс), забезпечуючи виконання потрібної реалізації методу на етапі виконання програми.

17) **Для чого можна використовувати інтерфейс як тип змінної?** Це дозволяє створювати універсальний, гнучкий код, який абстрагується від конкретних класів у кучі (Heap) та працює з усіма об’єктами, що реалізують відповідний інтерфейсний контракт (наприклад, `Connectable officeLaptop = new Laptop(...)`).

18) **Що відбувається під час виклику перевизначеного методу через змінну базового типу?** Завдяки механізму динамічного поліморфізму, під час виконання програми віртуальна машина Java (JVM) аналізує фактичний тип об’єкта, на який посилається змінна, і викликає саме перевизначену реалізацію дочірнього класу.

19) **Для чого в об’єктно-орієнтованому програмуванні використовують абстракцію?** Для виділення головних, спільних та визначальних характеристик об’єктів певної предметної області й відсікання другорядних деталей реалізації, що допомагає проектувати архітектуру складних систем.

20) **Які переваги дає використання успадкування та інтерфейсів під час розроблення програм?** Успадкування дозволяє уникати дублювання коду та будувати чіткі ієрархії пов’язаних класів, а інтерфейси дозволяють визначати спільні гнучкі правила взаємодії між різними класами, забезпечуючи легкість розширення та масштабування програми.

7 ІНФОРМАЦІЙНІ РЕСУРСИ ТА ПОСИЛАННЯ НА ВИХІДНИЙ КОД

Згідно з вимогами до здачі лабораторної роботи, повний проект розробленого вихідного коду додатку, розміщено у віддаленому репозиторії системи контролю версій.

– **Посилання на Git-репозиторій проєкту:**
`https://github.com/nasuehfw/oop-lab5.git`

– **Електронний каталог спільного доступу:**
https://drive.google.com/drive/folders/1oiJNrP62_ZFaXsxSDoPd4y1sXu0Bt5s1?usp=drive_link